



Universidad Nacional Experimental de Guayana.
Vicerrectorado Académico.
Coordinación General de Pregrado.
Proyecto de Carrera: Ingeniería en Informática.
Unidad Curricular: Ingeniería de Software II.

Gestión de Configuración, Versionamiento y DevOps.

Team Pixel & Code

“Estrategia, desarrollo y valor real”

Profesor:

Ms. Félix Márquez

Integrantes:

C.I.: V-28.530.717. Bravo Yvanna
C.I.: V-28.726.605. Guilarte Keibel
C.I.: V-19.505.397. Rivas Miguel
C.I.: V-24.848.424. Varguillas Génesis.

Ciudad Guayana, 22 de Septiembre del 2026.

Introducción

La Gestión de Configuración del Software (SCM, por sus siglas en inglés) surgió como una disciplina orientada a identificar, controlar y auditar los elementos que componen un sistema a lo largo de su ciclo de vida (Bersoff, 1984; IEEE, 2012). En su forma clásica, se apoyaba en planes documentales, líneas base formales y juntas de control de cambios que evaluaban cada modificación antes de autorizarla. Ese enfoque respondía a un contexto de lanzamientos espaciados y costosos, en el que cada cambio representaba un riesgo elevado.

La adopción de prácticas ágiles y del movimiento DevOps transformó ese escenario. Hoy los equipos integran código varias veces al día y despliegan de forma continua, lo que obliga a repensar cómo se ejercen el control, la trazabilidad y la auditoría (Kim et al., 2016). Las investigaciones de Forsgren et al. (2018) muestran que las organizaciones de alto rendimiento no renuncian al control de cambios, sino que lo automatizan. Para ello se apoyan en la integración continua, las pruebas automatizadas y la definición de la infraestructura como código (Humble y Farley, 2010; Morris, 2020).

El presente informe, aborda esta evolución desde cuatro ejes complementarios:

- En primer lugar, se diseña un flujo de trabajo en Git basado en GitHub Flow para un SaaS de facturación. Ese flujo resuelve de forma simultánea la publicación de una funcionalidad y la aplicación de un hotfix, e incluye una plantilla de Pull Request y quality gates automatizados.
- En segundo lugar, se comparan los modelos de ramificación GitFlow y Trunk-Based Development a la luz de la evidencia empírica de las métricas DORA.
- En tercer lugar, se analiza cómo el Pull Request, combinado con un pipeline de CI, sustituye a la Junta de Control de Cambios tradicional.
- Por último, se examinan los artefactos versionados (.gitignore, Dockerfile, workflows de CI y docker-compose.yml) que reemplazan al antiguo Plan de Gestión de Configuraciones. También se aborda el enfoque Everything as Code como respuesta al problema de la deriva de configuración.

El objetivo es demostrar que los principios clásicos de la SCM (identificación, control, registro de estado y auditoría) no desaparecen en la cultura DevOps. Esos principios se trasladan a mecanismos automatizados, ejecutables y verificables dentro del propio repositorio Git.

Pregunta 1: Flujo de Trabajo en Git y Pull Requests

Enunciado

Somos el líder técnico de un equipo que desarrolla un SaaS de facturación. Un desarrollador terminó la exportación a PDF en su rama local `feature/pdf-export` y, al mismo tiempo, aparece en producción un error que no deja iniciar sesión a los usuarios. Trabajando con GitHub Flow, hay que publicar la funcionalidad, aplicar el hotfix, redactar la plantilla del Pull Request y definir tres quality gates para el pipeline.

Diagrama de flujo

En GitHub Flow solo existe una rama permanente, `main`, y todo lo que está en ella se puede desplegar. El trabajo nuevo se hace en ramas cortas que vuelven a `main` por medio de un Pull Request, una práctica que el libro *Accelerate* relaciona con equipos que despliegan más seguido (Forsgren et al., 2018). Asumimos que `main` está protegida en GitHub: no admite push directo, y para hacer merge el PR debe pasar los quality gates, tener la aprobación de un compañero y estar al día con `main`. Cada merge dispara el despliegue. Con esas reglas armamos un flujograma para cada situación (Figuras 1 y 2).

a) Publicación de la funcionalidad

Antes de publicar nada, el desarrollador guarda su trabajo en un commit y actualiza la rama con lo último de `main` usando `git rebase`. Aquí el rebase no tiene riesgo porque la rama todavía no se ha subido al repositorio remoto, así que nadie más tiene esos commits. Después corre las pruebas en su máquina. Parece un paso de más, pero es mejor que el error aparezca en local y no en el PR, frente al revisor.

Con la rama publicada se abre el Pull Request en GitHub. La plantilla aparece sola en la descripción y el autor solo tiene que completarla. En ese momento arranca el pipeline con los quality gates, y en paralelo el revisor lee el código. Si alguno de los dos encuentra un problema, el desarrollador corrige, hace commit y push sobre la misma rama, y el PR se actualiza solo. No hace falta abrir uno nuevo.

La última condición antes del merge es que la rama esté al día con `main`. En nuestro caso

eso pasa de verdad, porque el hotfix entra a main mientras el PR de PDF sigue abierto. El desarrollador trae los cambios con `git fetch`, vuelve a hacer rebase y sube la rama con `git push --force-with-lease`. Si el rebase encuentra conflictos, se resuelven en los archivos, se marcan con `git add` y se sigue con `git rebase --continue`. Preferimos `--force-with-lease` antes que un push forzado común, porque se niega a sobrescribir la rama si alguien subió commits que el desarrollador no tiene en su copia local.

Para cerrar se usa Squash and merge, que junta los commits del PR en uno solo, y se borra la rama remota desde el mismo PR. Si la exportación da problemas en producción, se revierte con un único `git revert` de ese commit. Después el desarrollador cambia a main, la actualiza con `git pull` y recién ahí marca la versión v1.5.0, para que el tag quede sobre el commit que realmente está en producción. La rama local se borra con `git branch -D` y no con `-d`, porque después de un squash Git no reconoce la rama como integrada y `-d` se niega a borrarla.

b) Aplicación del hotfix

Lo primero es no mezclar el trabajo de PDF con la corrección. Si el desarrollador tenía cambios sin commit en su rama, los guarda con `git stash -u`. La opción `-u` incluye los archivos nuevos, que en una funcionalidad recién hecha seguramente existen y que un `git stash` normal dejaría afuera. La rama del hotfix se crea desde main, que es lo que está en producción, y en GitHub Flow es una rama corta más, igual que una feature. Si se creara desde `feature/pdf-export`, el código de PDF, que nadie ha revisado todavía, llegaría a producción junto con la corrección.

Antes de tocar el código escribimos una prueba que reproduzca el error. Al principio esa prueba falla, y cuando pasa sabemos que el problema quedó resuelto. Además queda en la suite, de modo que si el error vuelve a aparecer más adelante el pipeline lo va a detectar. Si no está claro qué commit introdujo el fallo, `git bisect` ayuda a encontrarlo con una búsqueda binaria en el historial.

El PR del hotfix pasa por los mismos quality gates que cualquier otro cambio. La urgencia no es excusa para saltarlos, porque un arreglo apurado sin pruebas puede romper otra cosa en producción. Lo que sí cambia es la prioridad: se marca con la etiqueta hotfix y el revisor lo atiende antes que el resto. Si el pipeline o el revisor piden cambios, se corrige sobre la misma rama y el PR se actualiza solo.

Después del merge se revisa en producción que los usuarios puedan entrar. Si el login sigue fallando, se analiza otra vez el problema y se repite el ciclo desde main, borrando antes la rama anterior para poder crearla de nuevo. Cuando está confirmado, el desarrollador actualiza su main local, marca la versión v1.4.1 y borra la rama. Por último, vuelve a feature/pdf-export, la reubica sobre main con rebase y recupera lo que había guardado con git stash pop. El orden importa: si recupera el stash antes del rebase, Git no permite hacer el rebase con cambios sin guardar.

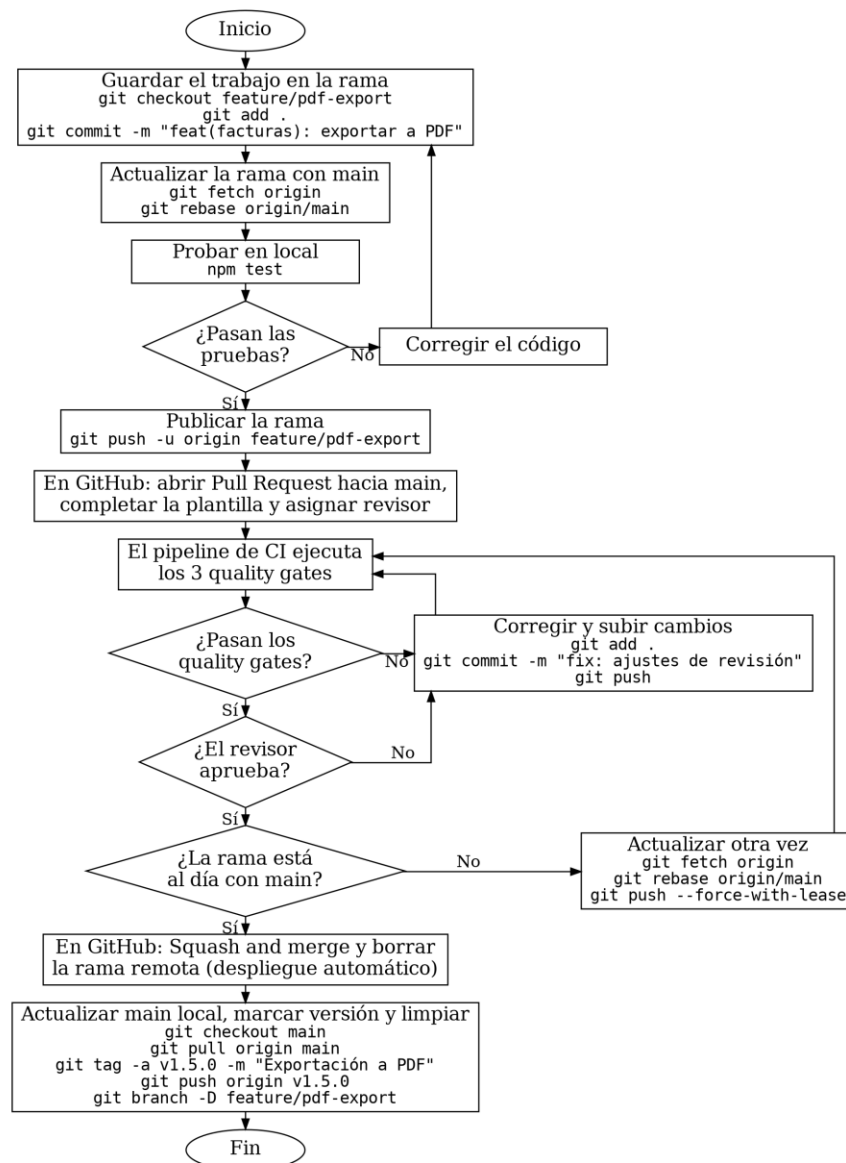


Figura 1. Flujograma para publicar la funcionalidad de exportación a PDF.

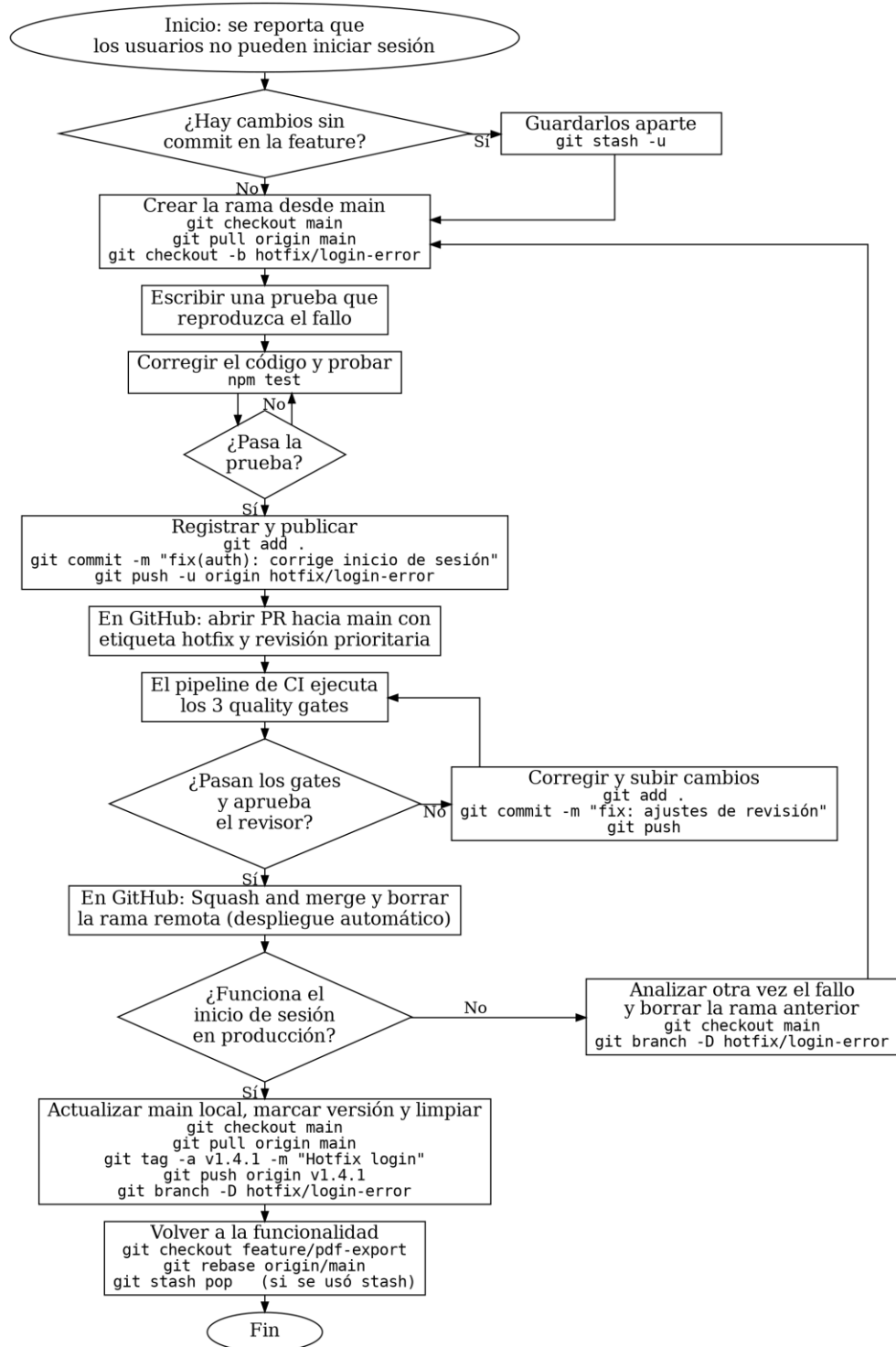


Figura 2. Flujograma para aplicar el hotfix del inicio de sesión.

Relación entre los dos flujos

La Figura 3 muestra cómo quedan las ramas en el tiempo. El hotfix sale de v1.4.0, se integra primero y genera v1.4.1. La rama de PDF había salido del mismo punto, pero antes de su merge se reubica encima de v1.4.1 con el rebase. Por eso la versión v1.5.0 trae las dos cosas, y el pipeline probó la exportación a PDF con la corrección del login ya incluida.

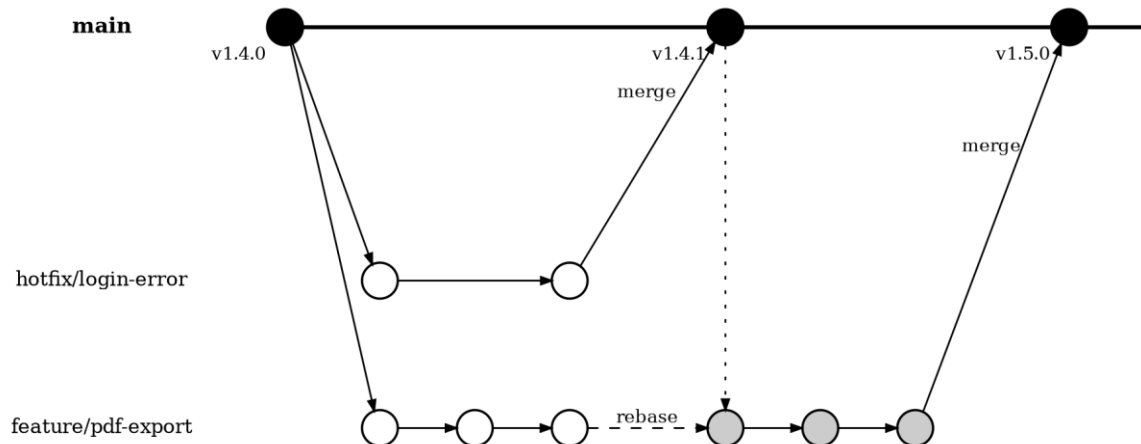


Figura 3. Ramas del repositorio durante el hotfix y la publicación de la funcionalidad.

En este modelo no hace falta `git cherry-pick`. En GitFlow sí se usa, porque además de **main** existen **develop** y las ramas de release, y la corrección tiene que llegar a todas ellas (Driessen, 2010). En GitHub Flow solo existe **main**, así que el arreglo entra una vez y las demás ramas lo reciben cuando se actualizan.

Diseño del Pull Request

La plantilla se guarda en el repositorio como `.github/pull_request_template.md` y GitHub la usa para llenar la descripción de cada PR nuevo. La mostramos ya completada para la exportación a PDF:

```
## Descripción
Agrega la opción de exportar una factura a PDF desde su vista de detalle.
Nuevo endpoint: GET /api/facturas/{id}/pdf
Ticket: [enlace]
```

```
## Tipo de cambio
- [x] Nueva funcionalidad
- [ ] Corrección de error
- [ ] Hotfix

## Impacto
- Módulo afectado: facturación.
- Nueva dependencia: librería de PDF [nombre y versión].
- Base de datos: sin cambios.
- El PDF contiene datos fiscales del cliente.
- Para revertir: git revert del commit que deja el squash en main.

## Cómo probar
1. Entrar con un usuario de prueba y abrir una factura.
2. Pulsar "Exportar PDF" y comparar montos e impuestos con la pantalla.
3. Pedir el PDF de una factura de otro usuario: debe responder 403.

## Checklist de revisión
- [ ] ¿Se agregaron pruebas unitarias?
- [ ] ¿Se revisó la seguridad de la librería de PDF?
- [ ] ¿Se ejecutó npm audit sin hallazgos altos ni críticos?
- [ ] ¿La versión quedó fijada en package-lock.json?
- [ ] ¿Un usuario solo puede exportar sus propias facturas?
- [ ] ¿Se escapan los datos del cliente antes de ponerlos en el PDF?
- [ ] ¿El lint pasa sin errores?
- [ ] ¿La rama está al día con main?
```

Las dos preguntas que pide el enunciado están en el checklist tal cual. Agregamos algunas más en la parte de seguridad porque este cambio tiene riesgos propios. Una librería de PDF procesa datos que vienen del usuario, y si tiene una vulnerabilidad conocida puede comprometer el servidor. Además, el endpoint recibe el número de factura en la URL. Si no se verifica que la factura pertenece a quien la pide, cualquier cliente podría descargar las facturas de otro con solo cambiar ese número.

Automatización (CI): quality gates

Un quality gate es una condición que el pipeline revisa de forma automática en cada PR.

En GitHub cada uno se marca como verificación obligatoria (required status check) en las reglas de protección de main. Mientras alguno esté en rojo, el botón de merge queda deshabilitado aunque el PR tenga la aprobación del revisor (Humble y Farley, 2010). Definimos estos tres:

1. **Pruebas y cobertura:** todas las pruebas unitarias y de integración deben pasar, y la cobertura de líneas no puede bajar del 80%. Lo configuramos en Jest con la opción `coverageThreshold`, que hace fallar la ejecución si no se llega al umbral. En un sistema de facturación un error en el cálculo de un impuesto se convierte en un problema legal para el cliente, por eso la cobertura no queda como algo opcional.
2. **Seguridad:** el análisis estático del código (SAST, con CodeQL) y la revisión de dependencias (SCA, con `npm audit --audit-level=high`) no pueden reportar vulnerabilidades altas ni críticas. `npm audit` termina con error si encuentra una, y eso hace fallar el trabajo. Para CodeQL se activa la regla `Require code scanning results` del repositorio, con el umbral en severidad alta o mayor, que impide el merge mientras exista una alerta de ese nivel. Este es el gate que detectaría una versión vulnerable de la librería de PDF antes de que llegue a producción.
3. **Build y estilo:** el proyecto se instala con `npm ci`, que usa exactamente las versiones del `package-lock.json`, el lint (ESLint) debe pasar sin errores ni advertencias y la imagen de Docker debe construirse sin fallas. Con esto nos aseguramos de que lo que se probó es lo mismo que se va a desplegar.

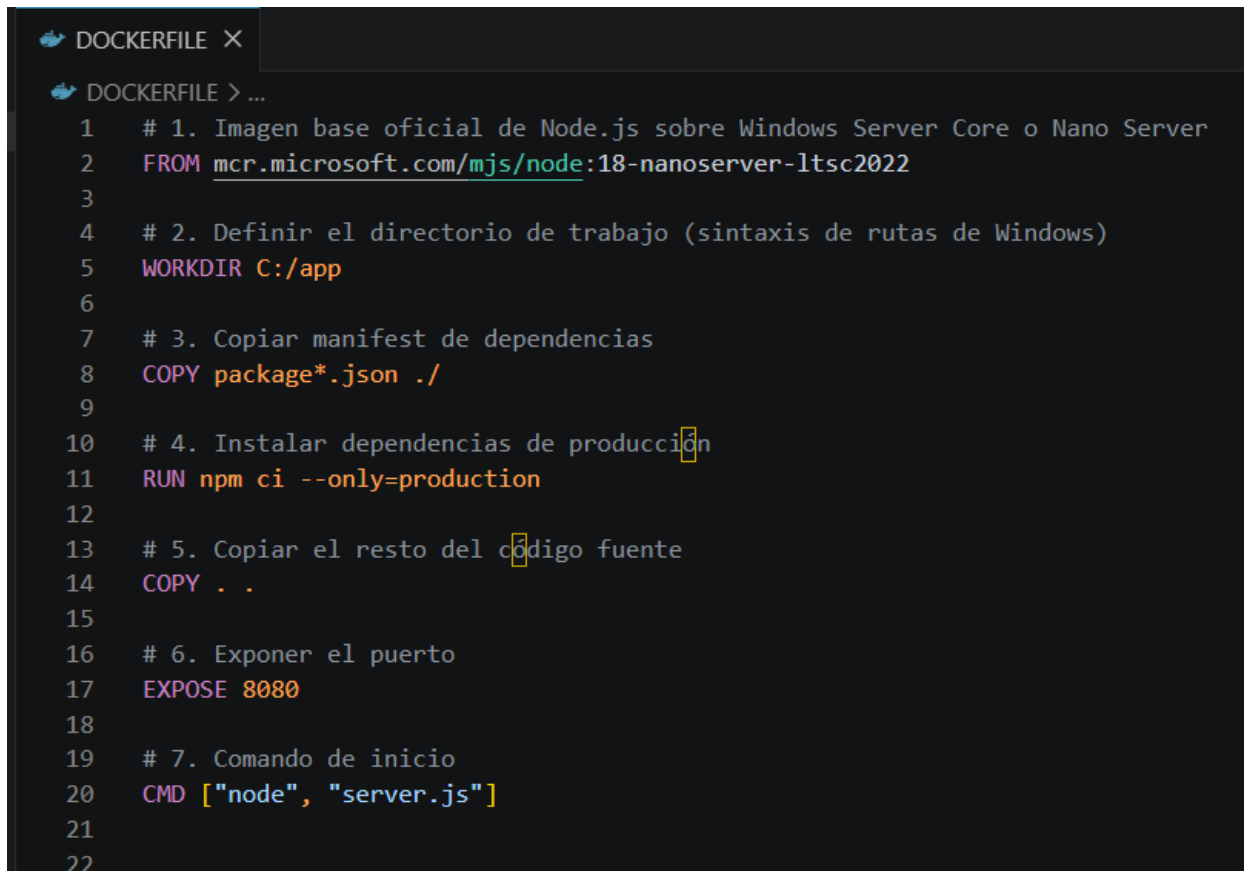
Los tres gates se definen como trabajos separados en el archivo `.github/workflows/ci.yml`. La aprobación del revisor no es un gate del pipeline, pero también es obligatoria para hacer el merge.

Ejercicio 2: Everything as Code y Gestión de Entornos Contexto:

Su empresa sufre del síndrome funciona en mi máquina. Los desarrolladores configuran sus entornos locales manualmente, y el equipo de operaciones instala las dependencias en los servidores de forma manual mediante scripts de Bash no versionados.

Instrucciones: Dockerización: Escriba un Dockerfile básico y comentado para una aplicación Node.js (asuma que el archivo principal es `server.js` y corre en el puerto 8080). Explique cómo

este archivo resuelve el problema de la deriva de configuración..



```
1  # 1. Imagen base oficial de Node.js sobre Windows Server Core o Nano Server
2  FROM mcr.microsoft.com/mjs/node:18-nanoserver-ltsc2022
3
4  # 2. Definir el directorio de trabajo (sintaxis de rutas de Windows)
5  WORKDIR C:/app
6
7  # 3. Copiar manifest de dependencias
8  COPY package*.json ./
9
10 # 4. Instalar dependencias de producción
11 RUN npm ci --only=production
12
13 # 5. Copiar el resto del código fuente
14 COPY . .
15
16 # 6. Exponer el puerto
17 EXPOSE 8080
18
19 # 7. Comando de inicio
20 CMD ["node", "server.js"]
21
22
```

El Dockerfile elimina este problema mediante los siguientes mecanismos:

1. **Entorno inmutable y declarativo:** En lugar de seguir una lista de pasos manuales que dependen del estado previo del servidor, el Dockerfile define exactamente la imagen base (versión de OS y Node.js), variables de entorno y dependencias requeridas.
2. **Determinismo en las ejecuciones:** Al usar una versión fija (como node:18-nanoserver) y comandos como npm ci (que instala las versiones exactas especificadas en package-lock.json), cualquier persona o servidor que construya la imagen obtendrá **un entorno idéntico byte por byte**.
3. **Aislamiento de dependencias:** La aplicación corre dentro de un contenedor aislado con su propio sistema de archivos y librerías, sin depender de los paquetes globales ni de las configuraciones del sistema operativo host donde se ejecute.

Reflexión: ¿Qué ventajas de trazabilidad y auditoría (conceptos clásicos de SCM) se obtienen al tratar la infraestructura como código en Git, en comparación con los logs manuales de un administrador de sistemas?

Considero podemos resumir estas ventajas en la siguiente tabla:

Criterio	IaC en Git	Logs / Acciones Manuales
Historial y Autoría	100% Trazable: Registro claro del <i>quién, qué y cuándo</i> (git log/blame).	Incompleto: Comandos SSH efímeros y logs locales borra bles.
Recuperación (Rollback)	Inmediato: Volver a un commit/tag previo restaura el entorno exacto.	Lento y Riesgoso: Revertir manualmente depende de la memoria o notas.
Control de Calidad	Revisión Obligatoria: Requiere aprobación de pares (<i>Pull Requests</i>).	Sin Control: Cambios directos en producción sin supervisión.
Documentación	Fuente Única de Verdad: El código describe el sistema real.	Obsoleta: Documentos en Wiki/Excel desconectados de la realidad.

En resumen, el uso de GIT constituye una acción responsable, para el desarrollo de software actual, en especial cuando los grupos de desarrollo incrementan en número y la trazabilidad consigo. Tener una herramienta que permita rastrear responsabilidades y acciones realizadas, permite un flujo de trabajo más eficiente y controlable según sea el caso.

Pregunta 2: Análisis de Modelos de Ramificación

Comparación entre GitFlow y Trunk-Based Development

La elección de un modelo de ramificación determina la frecuencia de integración, la complejidad operativa del equipo y la velocidad con la que el software puede llegar a producción. GitFlow, propuesto por Vincent Driessen (2010), fue diseñado en una época donde los ciclos de lanzamiento eran espaciados (semanas o meses) y el software se distribuía en versiones empaquetadas. Su estructura de ramas main, develop, feature, release y hotfix ofrece un control granular sobre qué se libera y cuándo, pero a costa de una integración tardía: el código de una funcionalidad puede vivir aislado en su propia rama durante días o semanas antes de fusionarse, acumulando divergencia respecto al resto del proyecto.

Trunk-Based Development invierte esta lógica. En lugar de aislar el trabajo, exige que todos los desarrolladores converjan constantemente sobre una única rama compartida, apoyándose en ramas de vida extremadamente corta (horas, no días) cuando se utilizan. Esta filosofía traslada el énfasis de "controlar el código antes de integrarlo" hacia "integrar constantemente y controlar la calidad mediante automatización". A continuación se presenta una comparación rigurosa entre ambos enfoques, considerando no solo su estructura de ramas, sino también su impacto operativo y organizacional:

Ramas	Múltiples ramas de larga vida (main, develop, feature, release, hotfix)	Una sola rama principal (trunk/main); ramas de vida muy corta (horas)
Frecuencia de integración	Baja y tardía; los cambios se acumulan antes de fusionar	Alta; varias integraciones al día
Complejidad operativa	Alta; estructura muy controlada y burocrática	Baja; flujo simplificado
Riesgo de conflictos	Alto ("merge hells") por integraciones tardías	Bajo; los cambios son pequeños y frecuentes

Caso de uso ideal	Software empaquetado, ciclos de release largos y predecibles	Entrega y despliegue continuo (SaaS)
Rol del desarrollador	Trabaja aislado en su rama durante días o semanas antes de integrar	Integra contra el trabajo de todo el equipo casi en tiempo real
Tamaño de los cambios integrados	Grande (features completas acumuladas)	Pequeño (commits incrementales, varias veces al día)
Dependencia de disciplina del equipo	Depende de que las ramas se mantengan cortas y sincronizadas manualmente	Depende de automatización robusta (CI, pruebas, feature flags)
Curva de aprendizaje	Alta; requiere entender múltiples flujos de ramas y su ciclo de vida	Baja en concepto, pero exige alta madurez técnica en pruebas y CI
Facilidad de rollback	Compleja si el error está mezclado con múltiples features en la misma release	Simple; los cambios pequeños son más fáciles de revertir o desactivar vía feature flag

Como se observa en la tabla, la diferencia más profunda entre ambos modelos no es meramente estructural (cuántas ramas existen), sino filosófica: GitFlow gestiona el riesgo mediante aislamiento y procedimientos, mientras que Trunk-Based Development lo gestiona mediante velocidad de retroalimentación y automatización. Esta distinción es la base de la argumentación que se desarrolla a continuación.

Argumentación basada en Accelerate (Forsgren et al., 2018)

El libro *Accelerate: The Science of Lean Software and DevOps*, escrito por Forsgren, Humble y Kim (2018), presenta evidencia empírica —recolectada a través de las métricas DORA (DevOps Research and Assessment) — que demuestra que el uso de Trunk-Based Development correlaciona directamente con un mayor rendimiento organizacional en la entrega de software.

Las organizaciones de alto rendimiento identificadas en el estudio presentan dos características

clave: una mayor frecuencia de despliegue y un menor tiempo de recuperación ante fallos (MTTR, Mean Time to Recovery). Ambas métricas están directamente relacionadas con la forma en que se gestionan las ramas de código:

- En GitFlow, las ramas de larga vida (feature, release) retrasan la integración. Los cambios se acumulan durante días o semanas antes de fusionarse en develop o main, lo que provoca que los conflictos e incompatibilidades se descubran tarde y en bloques grandes y difíciles de depurar.
- En Trunk-Based Development, cada desarrollador integra su código directamente contra el estado real y actualizado de la base compartida varias veces al día. Esto obliga a que los errores de compatibilidad se detecten de inmediato, en cambios pequeños y fáciles de aislar.

Un aspecto central del argumento de Accelerate es que la duración de las ramas de características (feature branch lifetime) es, por sí sola, uno de los predictores más fuertes del desempeño de un equipo de ingeniería. Cuanto más tiempo permanece un cambio fuera de la rama principal, mayor es la probabilidad de que diverja del trabajo de otros desarrolladores, lo que incrementa el costo de integrarlo posteriormente. Los autores documentan que los equipos de alto desempeño integran su código al menos una vez al día, una cadencia que resulta prácticamente incompatible con el ciclo de vida de las ramas feature y release de GitFlow, pero que es precisamente la norma en Trunk-Based Development.

Asimismo, Accelerate distingue entre estabilidad y throughput (rendimiento de entrega) como dos ejes que, contrario a la intuición tradicional, no son opuestos sino complementarios cuando existe automatización adecuada. GitFlow parte de la premisa de que aislar el código reduce el riesgo; sin embargo, la evidencia empírica muestra que aislar el código solo pospone el riesgo — lo concentra en el momento de la fusión— en lugar de eliminarlo. Trunk-Based Development, al forzar integraciones constantes, distribuye ese riesgo en unidades pequeñas y manejables, lo que en la práctica reduce tanto el tiempo de entrega como la tasa de fallos en los cambios (change failure rate), otra de las cuatro métricas clave de DORA.

Por esta razón, la industria de alto rendimiento se está moviendo hacia Trunk-Based Development: no porque reduzca el control sobre los cambios, sino porque sustituye el control

manual y esporádico (propio de GitFlow) por un control automatizado y constante, alineado con los principios de Integración Continua (CI) y Entrega Continua. En otras palabras, TBD no es "menos disciplina", sino disciplina de otro tipo: una disciplina delegada a sistemas automatizados en lugar de a procesos humanos secuenciales.

Prácticas de SCM obligatorias para que Trunk-Based no genere inestabilidad

Integrar código directamente en el trunk múltiples veces al día es una práctica de alto riesgo si no está respaldada por mecanismos automatizados que actúen como "redes de seguridad". A diferencia de GitFlow, donde el aislamiento de las ramas ofrece una protección estructural por defecto, en Trunk-Based Development esa protección debe construirse deliberadamente mediante procesos y herramientas. Las siguientes prácticas de SCM son, por tanto, condiciones obligatorias —no opcionales— para que el trunk se mantenga siempre en un estado desplegable:

a) Integración Continua (CI) automatizada

Cada vez que un desarrollador sube un cambio al trunk, un pipeline automatizado (por ejemplo, GitHub Actions, GitLab CI o Jenkins) debe ejecutar de forma inmediata pruebas unitarias, de integración y de contrato, además de análisis estático de código. Esto garantiza que cualquier incompatibilidad se detecte en minutos y no días después, cuando el costo de diagnosticarla y corregirla sería mucho mayor. La CI es, en esencia, el sustituto automatizado del control manual que ofrecía la separación de ramas en GitFlow.

b) Feature Flags (banderas de características)

Permiten subir al trunk código incompleto o en desarrollo, manteniéndolo oculto o inactivo para los usuarios finales mediante condicionales lógicos evaluados en tiempo de ejecución. De esta manera, el equipo puede integrar con alta frecuencia sin exponer funcionalidades a medio terminar, desacoplando el despliegue de código de la liberación de funcionalidades. Adicionalmente, los feature flags habilitan técnicas de despliegue progresivo como los canary releases o las pruebas A/B, permitiendo validar el comportamiento de una nueva funcionalidad en producción de forma controlada antes de exponerla a la totalidad de los usuarios.

c) Quality Gates (puertas de calidad)

Son umbrales automáticos —por ejemplo, un porcentaje mínimo de cobertura de pruebas, cero vulnerabilidades críticas detectadas o el cumplimiento de estándares de estilo de código— que bloquean automáticamente la fusión (merge) si no se cumplen. Esto evita que código de baja calidad llegue al trunk, incluso si un desarrollador lo intenta o lo hace por descuido, trasladando la responsabilidad de la calidad desde el juicio individual hacia una política objetiva y consistente aplicada a todo el equipo por igual.

d) Suite de pruebas automatizadas robusta

Sin un conjunto confiable de pruebas automatizadas (unitarias, de integración, de contrato y, en algunos casos, de extremo a extremo), no existe manera de verificar que el trunk sigue siendo funcional después de cada integración. Esta suite es la base que hace posible confiar en la alta frecuencia de cambios característica de TBD; sin ella, cada integración sería un acto de fe en lugar de una decisión respaldada por evidencia.

e) Revisión de código mediante Pull Requests de ciclo corto

Aunque Trunk-Based Development minimiza el uso de ramas de larga vida, esto no implica eliminar la revisión por pares. Por el contrario, exige Pull Requests pequeños y de ciclo de vida muy breve (horas, no días), de forma que la revisión de código no se convierta en un cuello de botella que retrase la integración, pero tampoco se sacrifique el control de calidad humano que complementa a la automatización.

f) Monitoreo y observabilidad post-despliegue

Dado que TBD favorece despliegues frecuentes e incrementales, resulta imprescindible contar con telemetría en tiempo real (métricas de error, latencia, uso de recursos) que permita detectar rápidamente cualquier degradación introducida por una integración reciente, habilitando una respuesta o rollback casi inmediato. Sin esta capacidad de observación, la velocidad de entrega de TBD se convertiría en una fuente de riesgo en lugar de una ventaja competitiva.

Anexo: Demostración práctica paso a paso

Con el fin de respaldar con evidencia el análisis teórico anterior, a continuación se documenta la secuencia exacta de comandos utilizada para simular ambos modelos de ramificación sobre el repositorio del grupo (Team Pixel & Code). Esta demostración puede reproducirse y verificarse directamente en el historial de commits del repositorio del Producto V.

Paso 1 — Preparar el repositorio base

Se parte siempre de la rama main actualizada:

```
git checkout main
git pull origin main
```

Paso 2 — Simular GitFlow: crear develop y una rama feature

Se crea la rama develop y, a partir de ella, una rama de funcionalidad de larga vida, en la que el trabajo puede permanecer aislado durante días:

```
git checkout -b develop
git checkout -b feature/reporte-mensual develop
# ... se edita el código de la funcionalidad ...
git add .
git commit -m "feat: agregar generacion de reporte mensual"
```

Paso 3 — Simular GitFlow: fusionar a develop y crear una release

Una vez terminada la funcionalidad, se integra a develop y se abre una rama de estabilización antes de llegar a producción:

```
git checkout develop
git merge --no-ff feature/reporte-mensual
git checkout -b release/1.1.0 develop
# ... pruebas de aceptacion y ajustes finales ...
```

Paso 4 — Simular GitFlow: liberar a main con una etiqueta de versión

La release se fusiona formalmente a main, se etiqueta la versión y se sincroniza nuevamente develop:

```
git checkout main
git merge --no-ff release/1.1.0
git tag -a v1.1.0 -m "Version 1.1.0: reporte mensual"
git checkout develop
git merge --no-ff release/1.1.0
git branch -d release/1.1.0
```

El historial resultante (visible con `git log --graph --oneline --all`) muestra un árbol ramificado, con múltiples líneas paralelas correspondientes a `develop`, `feature` y `release` — evidencia visual directa de la complejidad estructural de GitFlow descrita en la sección 1.

Paso 5 — Simular Trunk-Based Development: rama corta y fusión inmediata

Se parte nuevamente de `main` y se crea una rama que vive solo unas horas, protegida por un `feature flag`:

```
git checkout main && git pull
git checkout -b feature/boton-exportar
# cambio pequeño, oculto tras el flag EXPORT_ENABLED
git add .
git commit -m "feat: agregar boton de exportar (tras feature flag EXPORT_ENABLED)"
git checkout main
git merge feature/boton-exportar
git push origin main
git branch -d feature/boton-exportar
```

Paso 6 — Repetir la integración varias veces (simular el día a día)

El paso anterior se repite dos o tres veces más con cambios pequeños e independientes (por ejemplo, "fix: corregir texto del boton" o "feat: validar formato de fecha"), integrando cada uno directamente a `main`. El historial resultante muestra una línea prácticamente recta, con integraciones frecuentes y de bajo riesgo — el contraste visual directo frente al árbol de GitFlow del Paso 4.

Paso 7 — Verificar el disparo automático del pipeline de CI

Cada `push` a `main` descrito en los pasos 5 y 6 debe activar automáticamente el workflow definido en `.github/workflows/ci.yml`, ejecutando los tres Quality Gates (pruebas unitarias,

análisis estático y auditoría de dependencias). Un pipeline en verde tras cada integración es la evidencia práctica de que Trunk-Based Development únicamente es seguro porque existe automatización respaldándolo, tal como se argumentó en la sección 3.

Paso 8 — Comparar visualmente ambos historiales

Finalmente, se ejecuta el siguiente comando sobre cada uno de los dos historiales simulados:

```
git log --all --graph --oneline --decorate
```

La comparación de ambas salidas constituye la evidencia práctica más contundente del argumento central de esta pregunta: GitFlow produce un historial ramificado y complejo, mientras que Trunk-Based Development produce un historial lineal, con integraciones pequeñas y frecuentes, respaldadas por automatización en cada paso.

Pregunta 3. Evolución del Control de Cambios

Análisis de la afirmación

La afirmación según la cual “la Junta de Control de Cambios (CCB) tradicional es un cuello de botella que viola los principios de la Entrega Continua” es, en gran medida, correcta cuando se analiza desde la lógica operativa de los entornos de alta frecuencia de despliegue. La CCB nació en un contexto de SCM clásica, donde los cambios eran escasos, costosos y de alto riesgo, por lo que tenía sentido que un comité humano evaluara cada modificación antes de autorizarla. Sin embargo, este modelo presupone que la validación de un cambio requiere juicio humano centralizado y reuniones periódicas, lo cual es incompatible con equipos que integran código varias veces al día. La Entrega Continua exige que cualquier cambio que supere las validaciones automatizadas esté, en todo momento, en condición de ser liberado; si ese criterio depende de la disponibilidad de un comité que se reúne cada cierto tiempo, la velocidad de todo el flujo de trabajo queda subordinada al calendario de las personas y no a la calidad real del código.

No obstante, la crítica no implica que el control de cambios deba desaparecer, sino que debe transformarse en su naturaleza: de un filtro humano y periódico a un filtro automatizado, continuo y ejecutado en cada propuesta de cambio. En ese sentido, la afirmación es válida como

diagnóstico del problema, pero incompleta si se interpreta como una eliminación total de la gobernanza; lo que realmente ocurre es una descentralización y una automatización del control, no su desaparición.

Esta lectura no es solo una apreciación personal: Humble y Farley (2010) sostienen que la Entrega Continua exige que la base de código permanezca siempre en un estado desplegable, condición que un comité de reunión periódica no puede garantizar por diseño. En la misma línea, Forsgren et al. (2018), en su investigación empírica publicada en Accelerate, encuentran que los equipos de alto rendimiento correlacionan precisamente con la descentralización del control de cambios hacia mecanismos automatizados (CI robusto, despliegues frecuentes), y no con su eliminación. Ambas fuentes respaldan la idea de que lo que cambia no es la necesidad de gobernanza, sino su mecanismo de ejecución.

El Pull Request como Control de Cambios Automatizado

El Pull Request (PR), combinado con un pipeline de Integración Continua, sustituye funcionalmente a la CCB sin sacrificar la trazabilidad ni la auditoría que esta ofrecía. Cuando un desarrollador abre un PR, el sistema de control de versiones registra de manera inmutable quién propone el cambio, qué líneas se modifican, contra qué línea base se compara y en qué momento se realiza la solicitud; esta información, generada automáticamente, es más precisa y completa que un acta de reunión redactada manualmente por una CCB.

Sobre esa base, el pipeline de CI ejecuta de forma inmediata las verificaciones objetivas que antes dependían del criterio subjetivo del comité: pruebas unitarias y de integración, análisis estático de código (SAST), escaneo de vulnerabilidades en dependencias y validación de los Quality Gates definidos por el equipo. El resultado es binario y queda registrado en el propio repositorio: el cambio pasa o no pasa, y la razón exacta del fallo queda disponible para cualquier auditor futuro. A esta capa automatizada se suma la revisión por pares (Peer Review), donde uno o varios ingenieros evalúan aspectos que la máquina no puede juzgar por sí sola, como el diseño de la solución, su coherencia arquitectónica o su impacto en la seguridad del negocio.

Por qué este modelo es más seguro, rápido y auditable

- **Rapidez:** las validaciones automatizadas se ejecutan en minutos sobre cada propuesta de cambio, en lugar de esperar la próxima sesión de la CCB, lo que reduce drásticamente el

tiempo de espera entre que el código está listo y el momento en que puede integrarse.

- **Seguridad:** al ejecutarse en cada cambio y no de forma muestral, el pipeline aplica exactamente los mismos criterios de calidad a todo el código que ingresa al repositorio, eliminando la variabilidad e inconsistencia propia del juicio humano, que puede cambiar de una reunión a otra o entre distintos evaluadores.

- **Auditabilidad:** cada PR queda vinculado permanentemente a su historial de commits, a los resultados del pipeline y a los comentarios de revisión, de manera que el propio repositorio Git se convierte en el registro de auditoría, sustituyendo las actas y minutas en papel de la CCB por evidencia técnica verificable y reproducible.

Ejemplo práctico: plantilla de Pull Request y Quality Gates automatizados

Para que el PR funcione realmente como un control de cambios y no solo como un mecanismo de fusión de código, el equipo debe estandarizar su plantilla y definir Quality Gates objetivos en el pipeline. A continuación se muestra un ejemplo concreto de ambos artefactos, aplicados al caso de la funcionalidad de “exportación a PDF” mencionada en la Pregunta 1.

Ejemplo de plantilla de PR (.github/PULL_REQUEST_TEMPLATE.md):

```
## Descripción
Se implementa la funcionalidad de exportación de facturas a PDF
utilizando la librería pdftk. Incluye endpoint /api/invoices/:id/export
y botón de descarga en el panel de facturación.

## Impacto
- Módulos afectados: Facturación, API REST
- Nuevas dependencias: pdftk@0.14.0
- Riesgo estimado: bajo (funcionalidad aditiva, no modifica flujos existentes)

## Checklist de revisión
- [x] ¿Se agregaron pruebas unitarias?
- [x] ¿Se revisó la seguridad de la librería de PDF (CVEs conocidos)?
- [x] ¿Se actualizó la documentación (README / OpenAPI)?
- [ ] ¿Requiere migración de base de datos? (No aplica)
```

Ejemplo de Quality Gates en el pipeline (fragmento de .github/workflows/ci.yml):

```
jobs:
  quality-gates:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```
# Gate 1: pruebas unitarias y cobertura mínima
- name: Ejecutar pruebas y validar cobertura
  run: npm test -- --coverage --coverageThreshold='{ "global": { "lines": 80 } }'

# Gate 2: análisis estático de código (SAST)
- name: Analisis estatico (SAST)
  run: npx eslint . --max-warnings=0

# Gate 3: escaneo de vulnerabilidades en dependencias
- name: Escaneo de dependencias
  run: npm audit --audit-level=high
```

Con esta configuración, el botón de “Merge” en la plataforma solo se habilita si las tres etapas (pruebas con cobertura mínima del 80 %, cero advertencias del linter y ausencia de vulnerabilidades de severidad alta) terminan en estado exitoso, replicando así, de forma automática y consistente, el criterio que antes dependía del juicio manual de la CCB.

En conclusión, el control de cambios no se elimina en la cultura DevOps, sino que evoluciona de un mecanismo burocrático, manual y periódico a un mecanismo distribuido, automatizado y continuo, en el que el Pull Request y el pipeline de CI cumplen la misma función de gobernanza que la CCB, pero de forma más veloz, consistente y trazable. Como se desarrolla en la Pregunta 4, esta misma lógica de automatización no se detiene en el código de la aplicación: se extiende también a los artefactos que definen su entorno de ejecución, que dejan de vivir en documentos estáticos para convertirse en configuración versionada y sujeta al mismo control de cambios.

Pregunta 4. Artefactos y Auditoría

De la documentación en papel al código como documentación

En la SCM clásica, el “Plan de Gestión de Configuraciones” era un documento redactado en un procesador de texto, mantenido de forma manual y desconectado del propio código que pretendía gobernar. Esto generaba un riesgo constante de desactualización: el documento describía cómo “debía” configurarse el sistema, pero nada garantizaba que la configuración real coincidiera con lo escrito. En la era DevOps, ese plan se materializa en archivos de texto plano versionados dentro del mismo repositorio del proyecto, de manera que la documentación y la configuración son, literalmente, el mismo artefacto. Humble y Farley (2010) ya anticipaban este principio al definir la infraestructura y el proceso de despliegue como parte del mismo sistema versionado que el código fuente, idea que hoy se conoce como Everything as Code. A continuación se identifican, explican y ejemplifican cuatro de estos archivos.

1. .gitignore

Este archivo define qué rutas y patrones de archivos no deben ser rastreados por Git, evitando que artefactos temporales, dependencias generadas (como node_modules), credenciales

locales o archivos de configuración específicos de cada máquina se incorporen al historial del repositorio. Su función en la integridad del ítem de configuración es garantizar que el repositorio contenga únicamente el código fuente y los artefactos relevantes, evitando la contaminación de la línea base con archivos irreproducibles o sensibles. Al mismo tiempo, contribuye a la reproducibilidad, porque obliga a que cualquier archivo necesario para ejecutar el proyecto se genere de manera determinista a partir de instrucciones versionadas, en lugar de depender de archivos que un desarrollador pueda haber creado manualmente en su entorno local.

Ejemplo práctico (.gitignore para un proyecto Node.js):

```
# Dependencias generadas, se reconstruyen con npm install
node_modules/

# Variables de entorno y credenciales locales
.env
.env.local

# Artefactos de build
dist/
build/

# Archivos específicos del sistema operativo o del editor
.DS_Store
.vscode/
```

Sin este archivo, un desarrollador podría subir accidentalmente su carpeta `node_modules` (pesada e irreproducible tal cual) o su archivo `.env` con credenciales reales, contaminando la línea base y exponiendo información sensible en el historial de auditoría.

2. Dockerfile

El Dockerfile especifica, mediante instrucciones declarativas y secuenciales, cómo construir la imagen del entorno de ejecución de la aplicación: la imagen base, las dependencias del sistema operativo, la copia del código fuente, la instalación de librerías y el comando de arranque. Como ítem de configuración, su función central es la reproducibilidad determinista: cualquier persona o sistema que ejecute el mismo Dockerfile obtendrá una imagen funcionalmente idéntica, sin importar el entorno local donde se ejecute la construcción. Esto elimina la deriva de configuración entre desarrollo, pruebas y producción, y convierte al propio historial de Git del Dockerfile en el registro auditable de cómo ha evolucionado el entorno de ejecución de la aplicación a lo largo del tiempo.

Ejemplo práctico (Dockerfile para una app Node.js, servidor en `server.js`, puerto 8080):

```
# Etapa base: version fija de Node para reproducibilidad exacta
FROM node:20-alpine

WORKDIR /app

# Se copian primero solo los manifiestos de dependencias
# para aprovechar la cache de capas de Docker
```

```

COPY package*.json ./
RUN npm ci --omit=dev

# Luego se copia el resto del código fuente
COPY . .

# Buenas practicas de seguridad: usuario sin privilegios
USER node

EXPOSE 8080
CMD ["node", "server.js"]

```

Cualquier miembro del equipo, o el propio pipeline de CI, puede ejecutar docker build. Sobre este archivo y obtener exactamente la misma imagen, eliminando el clásico problema de “en mi máquina funciona”.

3. `.github/workflows/ci.yml`

Este archivo define el pipeline de Integración Continua: los eventos que lo disparan (por ejemplo, un push o un Pull Request), las etapas que se ejecutan (compilación, pruebas, análisis estático, escaneo de vulnerabilidades) y las condiciones bajo las cuales un cambio se considera apto para integrarse. Su aporte a la SCM moderna es doble: por un lado, automatiza y estandariza el control de cambios, sustituyendo el criterio manual de una CCB por Quality Gates objetivos y ejecutados de manera uniforme sobre cada propuesta de cambio; por otro lado, al estar versionado junto con el código, cualquier modificación al propio proceso de validación queda registrada, auditada y sujeta a revisión por pares, evitando que alguien altere silenciosamente los criterios de calidad del proyecto.

Ejemplo práctico (.github/workflows/ci.yml simplificado):

```

name: CI

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm ci
      - run: npm test
      - run: npm audit --audit-level=high

```

Si alguien intentara modificar este archivo para, por ejemplo, desactivar las pruebas antes de un cierre de sprint, ese cambio pasaría igualmente por un Pull Request, quedando visible,

comentado y auditado por el resto del equipo, algo imposible de lograr con un script de validación manual guardado en la máquina de un solo integrante.

4. docker-compose.yml (o manifiestos de infraestructura, por ejemplo Terraform)

El archivo docker-compose.yml describe de forma declarativa la topología completa de un entorno compuesto por varios servicios: qué contenedores deben levantarse, cómo se conectan en red, qué volúmenes persisten datos y qué variables de configuración utiliza cada servicio. Su función en términos de gestión de configuración es garantizar la reproducibilidad exacta de entornos completos y complejos con una sola instrucción, eliminando la necesidad de que cada desarrollador o el equipo de operaciones levante manualmente cada componente. Alternativamente, en escenarios de nube, archivos de Infraestructura como Código como los de Terraform cumplen una función equivalente a mayor escala: definen el estado deseado de la infraestructura, y su historial de cambios en Git permite auditar exactamente cuándo, por quién y por qué se modificó un recurso de producción.

Ejemplo práctico (docker-compose.yml para app + base de datos):

```
version: "3.9"
services:
  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      - DB_HOST=db
    depends_on:
      - db

  db:
    image: postgres:16-alpine
    environment:
      - POSTGRES_PASSWORD_FILE=/run/secrets/db_password
    volumes:
      - db_data:/var/lib/postgresql/data
    secrets:
      - db_password

volumes:
  db_data:

secrets:
  db_password:
    file: ./secrets/db_password.txt
```

Con un único comando (docker compose up) cualquier persona reconstruye el entorno completo (aplicación y base de datos, con su red y su volumen persistente) de forma idéntica en su máquina, en el entorno de pruebas o en producción, y cualquier cambio a esta topología queda registrado en el historial de Git del propio archivo.

Síntesis

Los cuatro artefactos anteriores comparten una misma lógica: convierten decisiones que antes vivían en la memoria de las personas o en documentos estáticos y desconectados del sistema, en instrucciones ejecutables, versionadas y sujetas al mismo proceso de control de cambios que el código de la aplicación. Esto garantiza tres propiedades simultáneamente: integridad, porque el estado descrito en el repositorio es, por construcción, el único estado válido y ejecutable del sistema; reproducibilidad, porque cualquier entorno puede reconstruirse de manera determinista a partir de estos archivos; y control, porque cada modificación a estos ítems de configuración queda registrada en el historial de Git, con autoría, fecha y justificación, convirtiendo al propio repositorio en el nuevo Plan de Gestión de Configuraciones, vivo y auditable.

En conjunto, las Preguntas 3 y 4 describen dos caras de un mismo fenómeno: la Pregunta 3 muestra cómo se automatiza la decisión de aceptar o rechazar un cambio, y la Pregunta 4 muestra cómo se automatiza la definición misma del entorno sobre el que ese cambio se ejecuta. Ambas dependen de un mismo principio de fondo: sustituir la memoria y el criterio manual de las personas por artefactos versionados, ejecutables y auditables en Git.

Referencias Bibliográficas

Bass, L., Weber, I., y Zhu, L. (2015). *DevOps: A software architect's perspective*. Addison-Wesley.

Bersoff, E. H. (1984). Elements of software configuration management. *IEEE Transactions on Software Engineering*, SE-10(1), 79–87.

Chacon, S., y Straub, B. (2014). *Pro Git* (2.^a ed.). Apress. <https://git-scm.com/book/es/v2>

Conventional Commits. (s.f.). *Conventional Commits 1.0.0*. <https://www.conventionalcommits.org/es/v1.0.0/>

Docker Inc. (s.f.). *Dockerfile reference*. Docker Docs. <https://docs.docker.com/reference/dockerfile/>

DORA. (2024). *Accelerate State of DevOps Report 2024*. Google Cloud. <https://dora.dev/research/2024/dora-report/>

Driessen, V. (2010). *A successful Git branching model*. nvie.com. <https://nvie.com/posts/a-successful-git-branching-model/>

Forsgren, N., Humble, J., y Kim, G. (2018). *Accelerate: The science of lean software and DevOps*. IT Revolution Press.

Fowler, M. (2006). *Continuous integration*. martinowler.com.
<https://martinfowler.com/articles/continuousIntegration.html>

Fowler, M. (2020). *Patterns for managing source code branches*. martinowler.com.
<https://martinfowler.com/articles/branching-patterns.html>

GitHub. (s.f.). *GitHub flow*. GitHub Docs. <https://docs.github.com/es/get-started/using-github/github-flow>

GitHub. (s.f.). *About protected branches*. GitHub Docs.
<https://docs.github.com/es/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches>

Hammant, P. (s.f.). *Trunk based development*. <https://trunkbaseddevelopment.com>

Hodgson, P. (2017). *Feature toggles (aka feature flags)*. martinowler.com.
<https://martinfowler.com/articles/feature-toggles.html>

Humble, J. (2018). Continuous delivery sounds great, but it won't work here. *Communications of the ACM*, 61(4), 34–39.

Humble, J., y Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.

IEEE. (2012). *IEEE Std 828-2012: IEEE standard for configuration management in systems and software engineering*. IEEE.

Kim, G., Humble, J., Debois, P., y Willis, J. (2016). *The DevOps handbook: How to create world-class agility, reliability, and security in technology organizations*. IT Revolution Press.

Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239).

Morris, K. (2020). *Infrastructure as code: Dynamic systems for the cloud age* (2.^a ed.). O'Reilly Media.

OWASP Foundation. (2021). *OWASP Top 10: 2021*. <https://owasp.org/Top10/es/>

Preston-Werner, T. (s.f.). *Versionado semántico 2.0.0*. <https://semver.org/lang/es/>

Pressman, R. S. (2010). *Ingeniería del software: Un enfoque práctico* (7.^a ed.). McGraw-Hill.

Sommerville, I. (2011). *Ingeniería de software* (9.^a ed.). Pearson Educación.